PROJECT PRESENTATION

FPGA-Accelerated LMS Beamforming

GNU Radio • PYNQ/Zynq • USRP • AXI DMA integration

Reference paper

PYNQ hardware-in-the-loop architecture for radar acceleration

Our adaptation

Replace CAF with LMS adaptive beamforming, then move toward FPGA acceleration

Current status

Pipeline verified; final debugging focused on AXI DMA streaming

ALGORITHM CHOICE

Why LMS: a controlled first step before QRD-RLS or full MIMO

LMS adaptive update

$$y(k) = w^H(k)\, x(k)$$
$$e(k) = d(k) - y(k)$$
$$w(k+1) = w(k) + \mu\, x(k) e^*(k)$$

Normalized version used in the Python server:
w ← w + (μ / power) · x · conj(e)

1 Simple datapath

LMS maps naturally to MAC operations, subtraction, complex multiplication and weight registers.

2 Good for integration

It lets us verify packets, fixed-point format and DMA before implementing heavier algorithms.

3 Bridge to beamforming

The adaptive weights are the key concept that later supports receive beamforming and MIMO processing.

4 Scalable roadmap

After LMS works, QRD-RLS or multi-antenna beamforming can reuse the same host-PYNQ interface.

Positioning statement

QRD-RLS is the advanced objective, but LMS is the practical first hardware milestone because it validates the same adaptive-weight concept with a lower-risk FPGA datapath.

STORYLINE

Project narrative: from RF theory to a hardware-in-the-loop prototype

The project started as a MIMO/beamforming system study, then evolved into a practical FPGA-based signal processing pipeline.

1 RF problem

Receive/transmit chains must be configured, synchronized and calibrated before meaningful beamforming.

2 Software baseline

GNU Radio flowgraphs, USRP settings and LMS logic were used to understand signals and debug behavior.

3 PYNQ bridge

UDP communication proved that the host can send complex data to the board and receive results.

4 FPGA path

.bit/.hwh files, AXI GPIO, DMA and Q1.15 packing moved the system toward hardware acceleration.

5 Final demo

Live SDR input, FPGA LMS processing and GNU Radio visualization.

! Core challenge

The hardest part is not only the LMS equation; it is the full system integration: USRP data, network transfer, PYNQ control, fixed-point formatting and DMA streaming.

↧ Engineering approach

We isolated each layer before connecting everything: test signals first, then UDP, then overlay, then AXI GPIO, then DMA, then USRP.

✓ Current position

Most infrastructure layers are validated. The remaining technical focus is stable AXI-Stream/DMA execution of the FPGA LMS core.

Reference paper: hardware-in-the-loop with GNU Radio and PYNQ

The paper demonstrates a practical workflow where GNU Radio remains on the host computer, while PYNQ/Zynq acts as a processing node connected through UDP.

Host PC
GNU Radio
GUI + signals

→

UDP Client
custom block sends
IQ vectors + config

→

PYNQ/Zynq
ARM processor runs
UDP server + driver

→

FPGA Core
CAF accelerator
in programmable logic

→

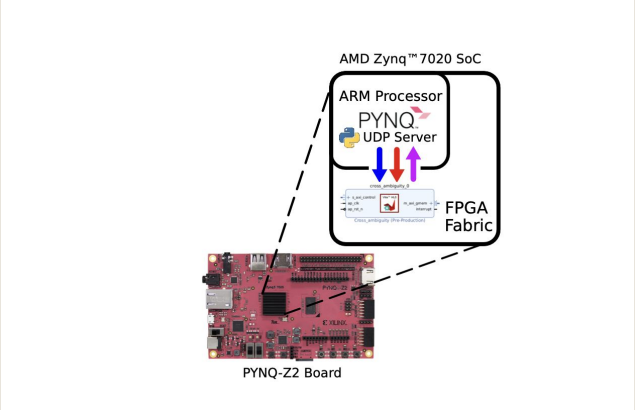
Result
Range-Doppler
visualized

Key idea we reused

Do not force GNU Radio GUI to run on PYNQ. Keep heavy visualization on the laptop and use PYNQ as an embedded processing server.

Original algorithm

The paper accelerates the Cross Ambiguity Function (CAF) for radar processing and returns a range-Doppler style output.



OUR CONTRIBUTION

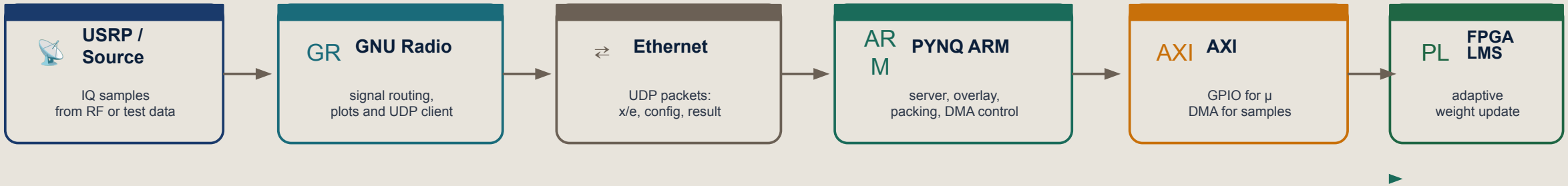
What we changed: from CAF radar processing to LMS adaptive beamforming

We did not copy the paper algorithm. We copied its integration strategy and replaced the processing core with an LMS path that fits our MIMO/beamforming goal.

Dimension	Reference paper	Our project
Application	Radar hardware-in-the-loop simulation	MIMO/beamforming and adaptive filtering prototype
Algorithm	CAF / Cross Ambiguity Function	LMS adaptive filter first; QRD-RLS or larger MIMO beamforming later
Data path	GNU Radio UDP client → PYNQ server → FPGA core	Same communication skeleton, then LMS processing backend
Output	Radar map / processed radar result	LMS output, error magnitude and adaptive weights
Reason for staging	Accelerate a known radar computation	Validate integration before adding complex FPGA datapaths

END-TO-END SYSTEM

Target system architecture with LMS in the loop



Host side

Generates or receives IQ data, builds x and e test streams, sends packets and visualizes LMS behavior.

PYNQ side

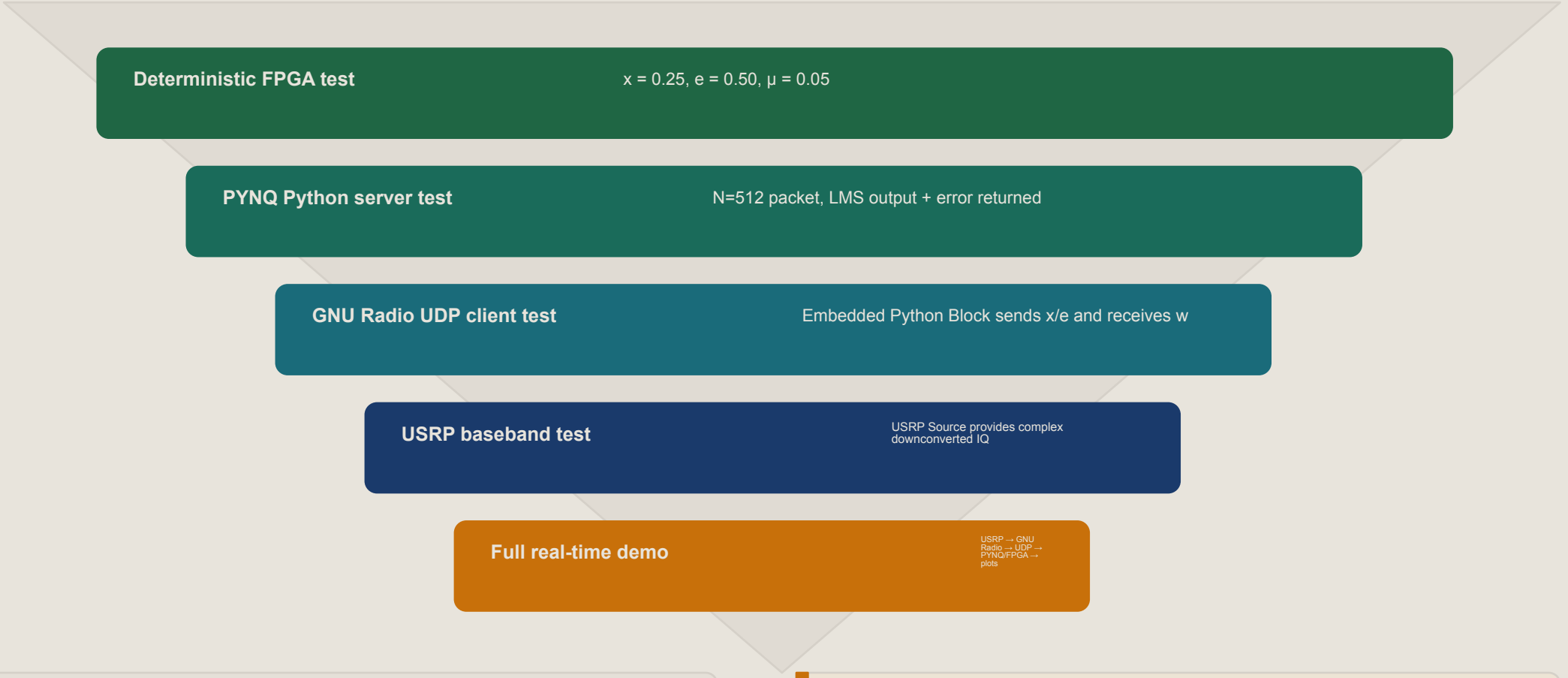
Loads overlay, receives UDP packets, converts float data into Q1.15 format, controls DMA and GPIO.

FPGA side

Executes the LMS update in deterministic hardware once the AXI-Stream path is stable.

VALIDATION SEQUENCE

Testing path: from controlled numbers to real RF data



Current best test

Use constants first to make GNU Radio and Jupyter comparable.

Why not USRP first?

RF adds gain, antenna, sync and cable variables before the FPGA path is stable.

RF INTEGRATION

USRP/GNU Radio lessons learned

IQ

USRP Source is already baseband IQ

The UHD Source gives complex downconverted samples. We do not need to manually downconvert RF again before LMS testing.

2x

Single vs dual USRP

Single-USRP loopback is simpler. Dual-USRP tests need matching frequency, gain, sample rate and stable network/device addressing.

RX

Valid antenna ports matter

UBX RX ports are not arbitrary labels. Wrong selection stops the flowgraph before signal processing begins.

How we interpret plots

- A flat zero output can mean no data, disconnected ports, timeout, wrong response format, or the server returning zeros after an exception.
- A “too perfect” signal may come from direct cabling, very low noise, or synthetic sources rather than a real channel.
- Real LMS behavior should show changing weights and an error curve that reduces or stabilizes.

Conclusion: USRP should be integrated only after the deterministic GNU → PYNQ → FPGA path is stable.

FPGA integration: what works and what is still under debugging

Verified working

Overlay loads; IP dictionary shows axi_dma_0, axi_gpio_0 and processing_system7_0; μ GPIO communication is accessible; Q1.15 packing format defined.

Under debugging

DMA execution fails with “DMA channel not started”, which points to the stream interface rather than the host-side packet logic.

Likely checks

TVALID/TREADY handshake, TLAST propagation, input/output width, DMA simple mode, reset, clock domain and IP control signals.

Key status: hardware files and control path are visible; the remaining blocker is stable AXI-Stream/DMA movement through the LMS core.

Data format and deterministic FPGA test case

Q1.15 fixed-point format

```
float → int16 = round(value × 32768)

   x = 0.25 → 0x2000
   e = 0.50 → 0x4000
packed word = 0x20004000
```

32-bit stream word layout

GR GNU Radio test input

Constant Source x = 0.25 and e = 0.50 first. This avoids RF variability and should match Jupyter.

PY PYNQ server action

Receive float32 packet, convert to Q1.15, configure μ through GPIO, then trigger DMA transfer.

✓ Expected result

The same deterministic input should produce repeatable w_out values in both Jupyter and GNU Radio.

AXI Protocol

AMBA Family

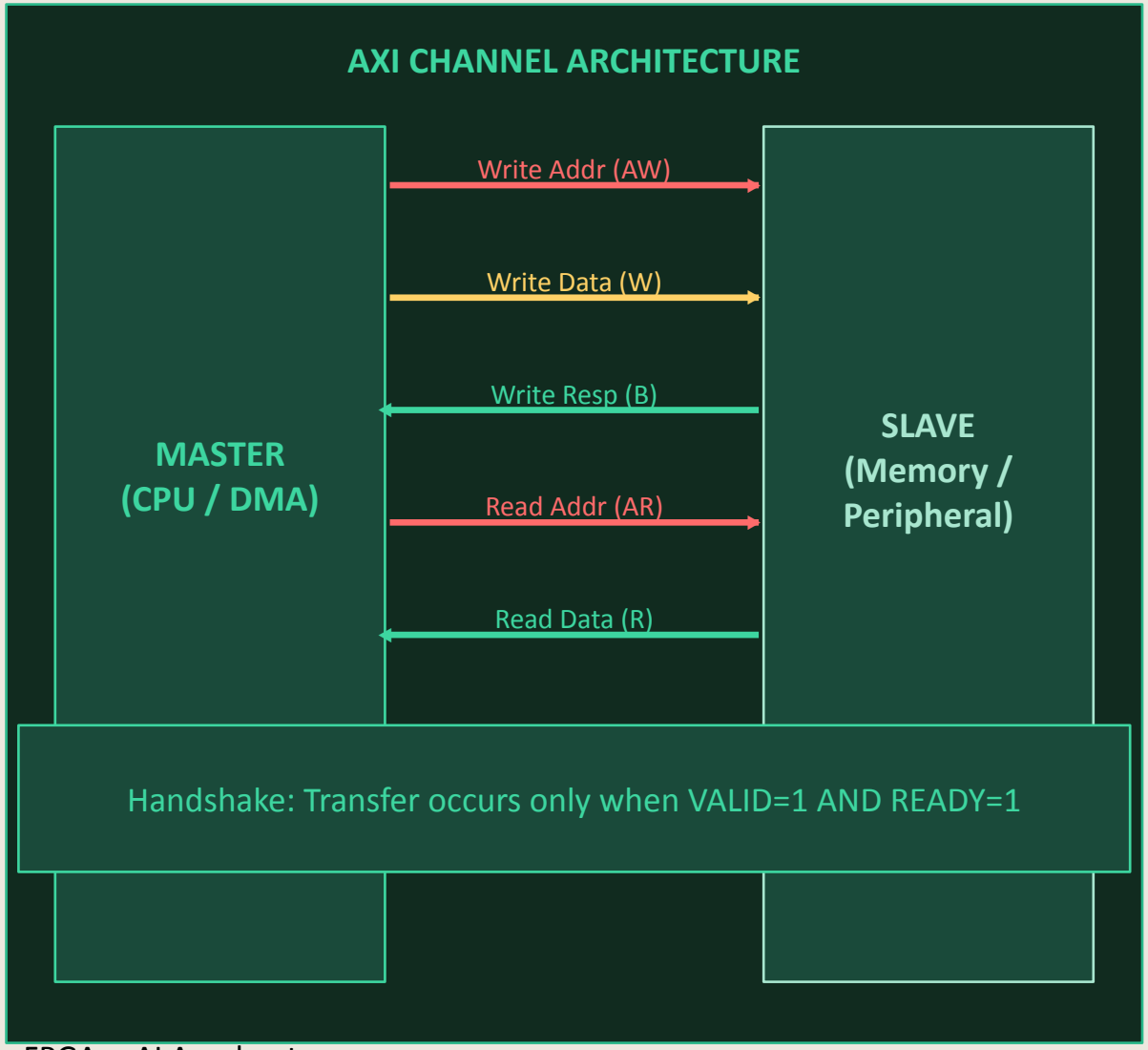
Advanced eXtensible Interface

WHAT IS AXI?

AXI is part of ARM's AMBA protocol family. It enables high-bandwidth, low-latency communication between processors, memory, DMA engines, and peripherals inside SoCs and FPGAs.

WHY USE AXI?

- High bandwidth & parallel data transfer
- Separate read / write channels (no deadlock)
- Burst transfers — up to 256 beats
- Out-of-order transactions via transaction IDs
- Industry standard — portable across vendors

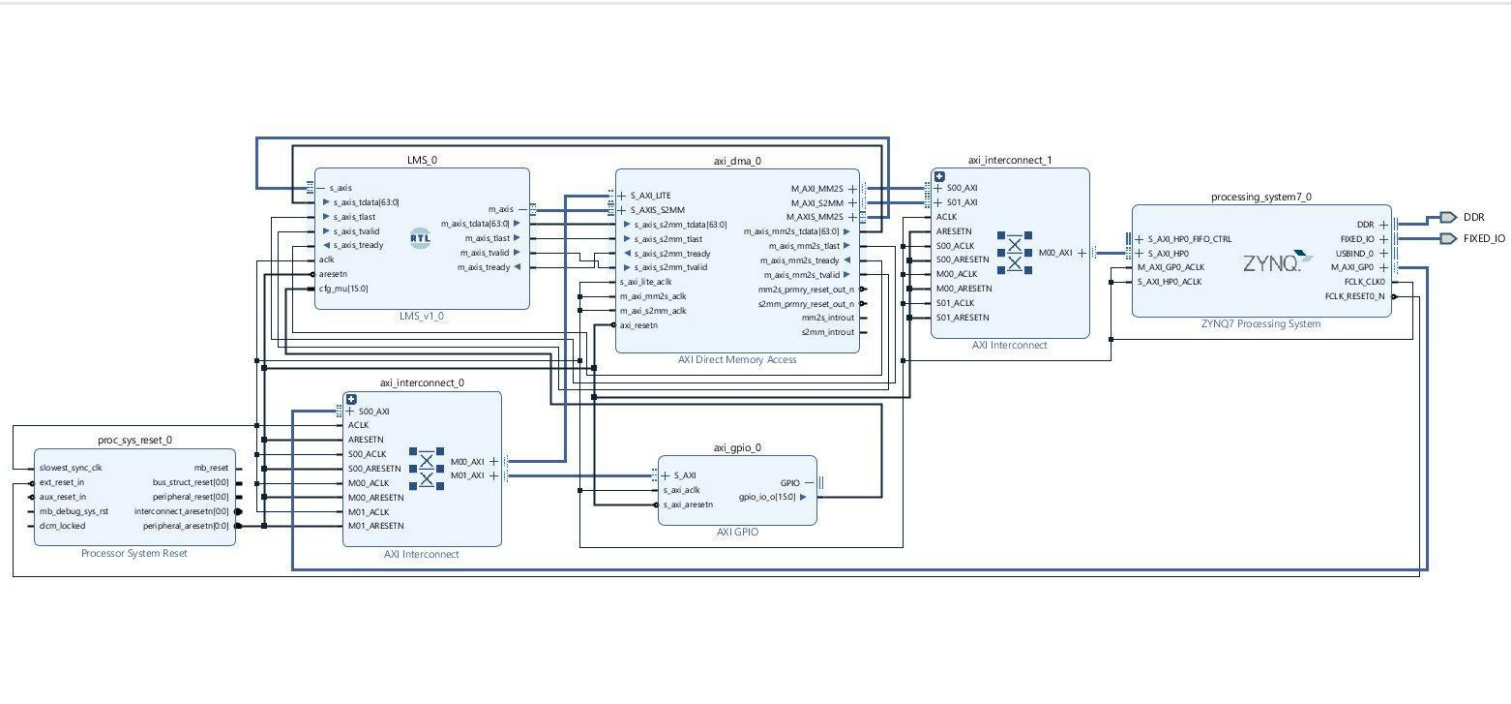


AXI4	AXI-Lite	AXI-Stream
Full feature	Simple registers	Data streaming

Used in: SoCs · CPUs · GPUs · FPGAs · AI Accelerators

LMS Adaptive Filter — FPGA IP Block Diagram

4-Tap Complex LMS · 14-Stage Pipeline · Zynq-7000 SoC



KEY IP BLOCKS

LMS_v1_0 (RTL Core)

4-tap complex LMS, 14-stage pipeline, 16 DSPs

AXI DMA

AXI-Stream bridge; DMA data flows to DDR via PS HP0 port

AXI GPIO

Delivers step-size cfg_mu[15:0] from PS to LMS core

AXI Interconnect ×2

IC0: control bus (GP0); IC1: HP0 data path to DDR

Proc Sys Reset

Fans out synchronised aresetn — no AXI data path

ZYNQ-7 PS

ARM host, DDR controller, and FIXED_IO (all PS external pins)

HARDWARE RESULTS · FPGA ACCELERATOR

1.5197

HW THROUGHPUT (MSAMPLES / S)

0.674 ms

AVG DMA + FPGA TIME PER CHUNK

0.202 s

TOTAL HARDWARE TIME (RAW COMPUTE)

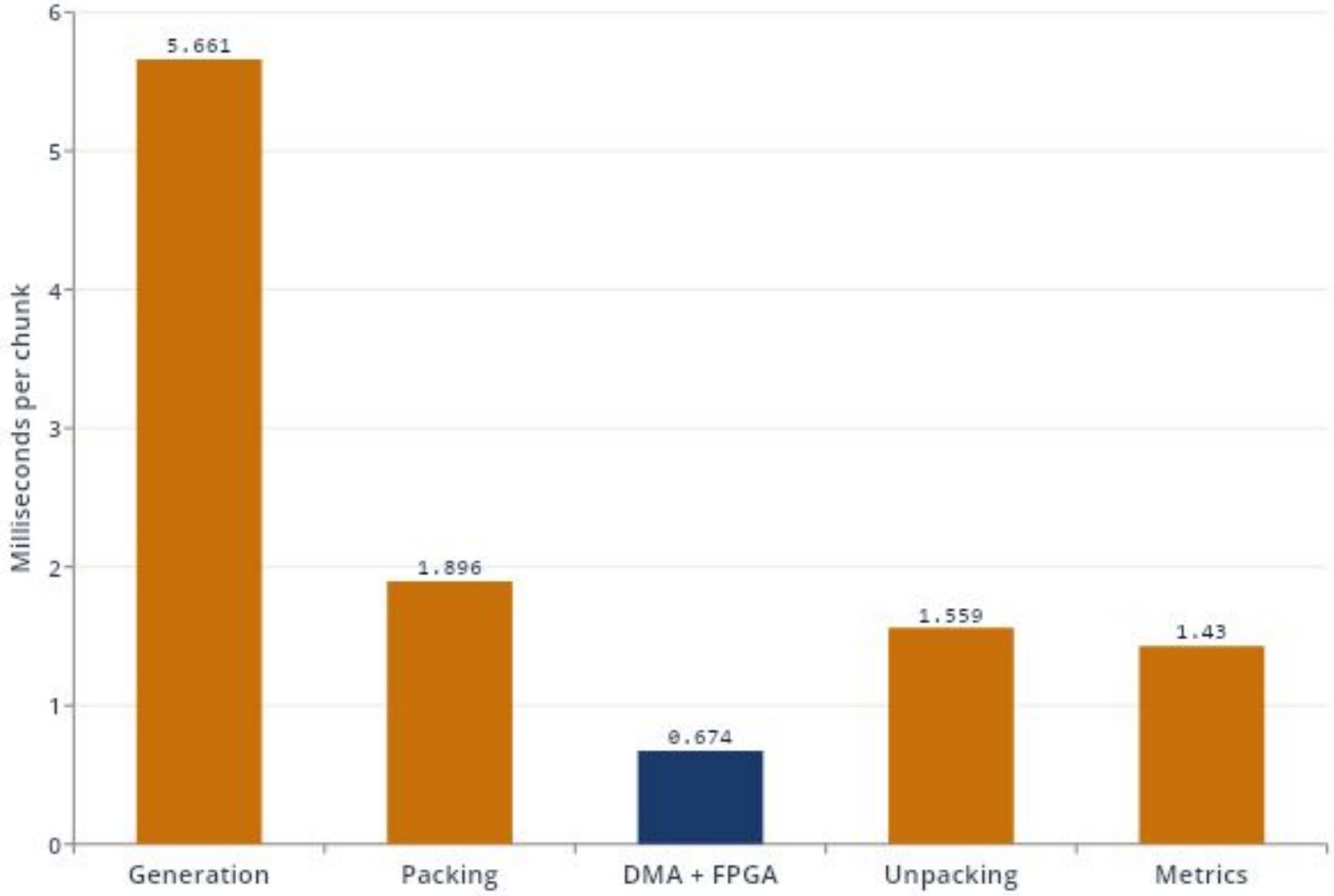
3.763 s

TOTAL SCRIPT TIME (END-TO-END)

THE ACCELERATOR SPENDS ONLY ~5% OF WALL-CLOCK TIME IN ACTUAL HARDWARE COMPUTE.

Remaining time is host-side: NumPy generation, byte packing, NumPy metric evaluation. These overheads dominate the pipeline.

PER-CHUNK TIMING BREAKDOWN



OBSERVATION

Total avg processing: `12.199 ms / chunk`

Pure HW compute (DMA + FPGA) is **0.674 ms — only 5.5% of the pipeline.**

Generation (5.661 ms) and packing (1.896 ms) are the dominant cost — both are host-side preparation work, not accelerator limitations.

IMPLICATION

Further speedup requires moving generation & metric computation either onto the FPGA or onto a separate host thread overlapped with DMA.

SOFTWARE BASELINE · CPU REFERENCE IMPLEMENTATION

0.0087

SW THROUGHPUT (MSAMPLES / S)

116.979 ms

AVG PROCESSING TIME PER CHUNK

35.161 s

TOTAL SCRIPT TIME (END-TO-END)

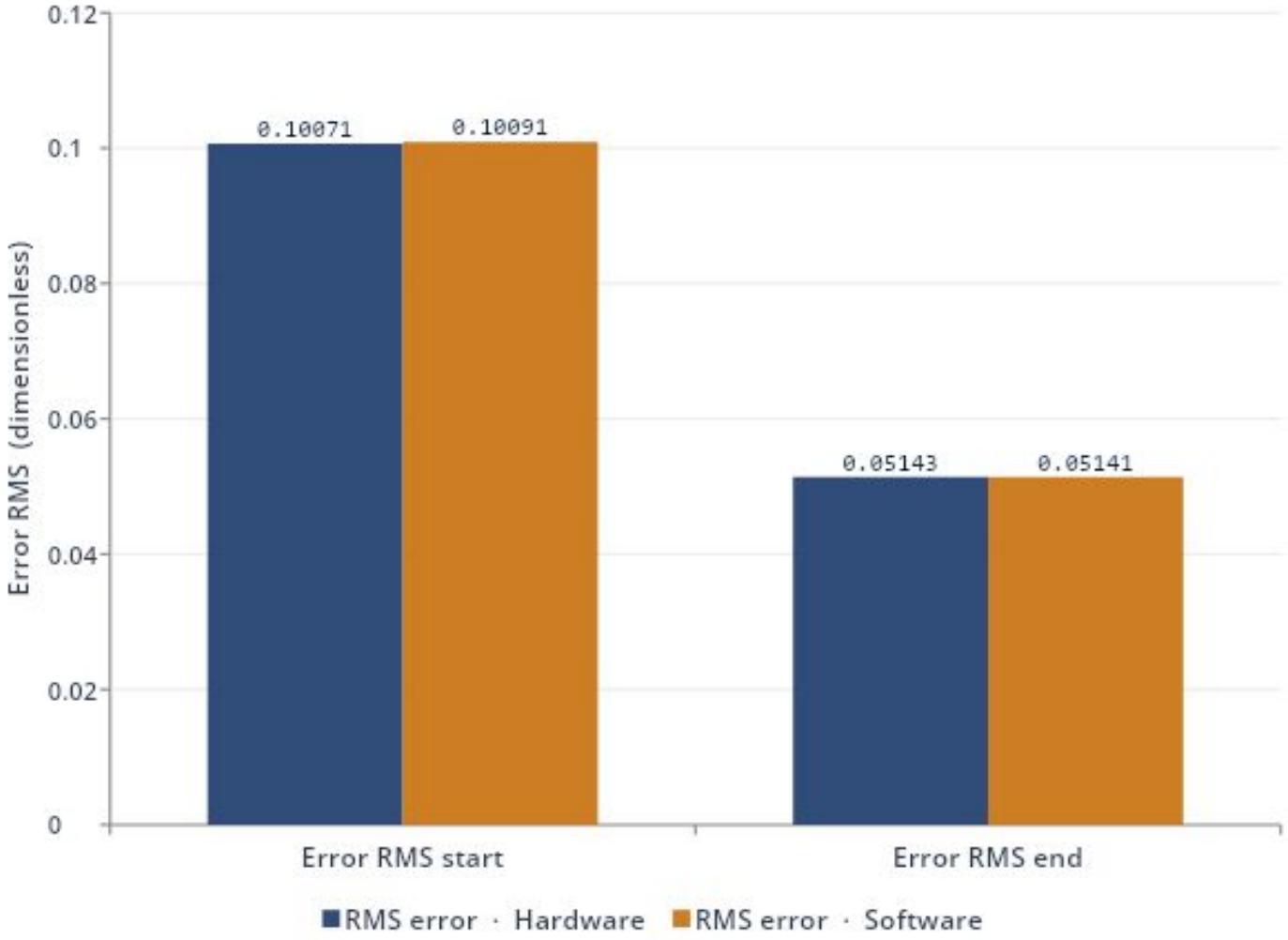
307,200

SAMPLES PROCESSED (IDENTICAL TO HW)

REFERENCE PATH IS A SINGLE-THREADED NUMPY LOOP — NO VECTORISATION, NO PARALLELISM.

Used as the functional ground-truth for the FPGA implementation — bit-for-bit comparable up to numerical precision.

LMS CONVERGENCE · SIGNAL QUALITY EQUIVALENCE



49.1 %
ERROR REDUCTION

6.04 dB
FINAL SNR

0.21313
1-TAP ISI FLOOR (THEO)

LEARNED WEIGHT

h_0 (target) **0.600 + 0j**
w (learned) **0.840 + 0j**
$|w - h_0|$ **0.24049**

HARDWARE vs SOFTWARE · HEAD-TO-HEAD

CPU · SOFTWARE (NUMPY)		FPGA · HARDWARE (STREAM)	
Total script time	35.161 s	Total script time	3.763 s
Throughput	0.0087 MS/s	Throughput (HW pure)	1.5197 MS/s
Avg processing / chunk	116.979 ms	Avg processing / chunk	12.199 ms
Parallelism	scalar, sequential	Parallelism	pipelined stream

9.3× end-to-end · **175×** raw compute · *identical convergence*